

Advanced AI Answer Grading System

Technical Documentation

Using Sentence-BERT, SpaCy, and Advanced NLP

Submitted by:

Tanmay Kumar Shrivastava

December 15, 2025

*An Automated Answer Evaluation System
Powered by Machine Learning and Natural Language Processing*

Version 2.0 - Improved Feedback System
With Human-like, Exam-specific Assessment

Contents

Abstract	3
1 Introduction	4
1.1 Overview	4
1.2 Key Features	4
1.3 System Architecture	4
2 Technical Stack	5
2.1 Core Libraries and Dependencies	5
2.2 NLP Models	5
3 System Components	6
3.1 Advanced Grading Engine	6
3.1.1 Initialization	6
3.1.2 Key Concept Extraction	6
3.1.3 Semantic Similarity Calculation	7
3.1.4 Answer Quality Analysis	7
3.1.5 Keyword Coverage Analysis	7
3.1.6 Point Grading Algorithm	8
3.2 Enhanced Feedback Generator (Human-like)	9
3.2.1 Point-Specific Feedback Generation	9
3.2.2 Overall Feedback Generation	10
3.3 Scoring System - KEY CHANGE	12
3.4 PDF Report Generator	12
3.4.1 Report Structure	12
4 Interactive User Interface	13
4.1 UI Components	13
4.2 Grading Process Flow	13
4.3 Model Answer Format	13
5 Technical Specifications	14
5.1 Model Parameters	14
5.2 Performance Metrics	14
6 System Advantages	15
6.1 Benefits Over Traditional Grading	15
6.2 Educational Value	15
7 Key Improvements in Version 2.0	16
7.1 Partial Marking Policy	16
7.2 Feedback Quality	16
7.3 Exam-Focused Language	16
8 Limitations and Future Work	17
8.1 Current Limitations	17
8.2 Proposed Enhancements	17
9 Conclusion	18
9.1 Key Achievements	18
9.2 Impact on Education	18

9.3 Version 2.0 Summary	18
Acknowledgments	20
Appendix A: Sample Code Snippets	20
Appendix B: Usage Instructions	24
Appendix C: Example Assessment	25

Abstract

This document presents a comprehensive technical documentation of an Advanced AI Answer Grading System that employs state-of-the-art Natural Language Processing (NLP) techniques for automated assessment of student answers. The system utilizes Sentence-BERT for semantic similarity analysis, SpaCy for linguistic processing, and generates specific, human-like evaluation feedback.

The grading system analyzes student responses across multiple dimensions including semantic understanding, keyword coverage, concept identification, and writing quality metrics. It generates exam-focused feedback with specific recommendations, mimicking the evaluation approach of experienced educators.

Key Improvement: This version implements a human-like feedback system where partial answers receive no marks and students are directed to consult with markers for clarification. All feedback is specific to the question content rather than generic AI-generated comments.

1 Introduction

1.1 Overview

The Advanced AI Answer Grading System is an intelligent educational tool designed to automate the assessment of descriptive answers using cutting-edge artificial intelligence and natural language processing technologies. Traditional manual grading is time-consuming and subject to human bias; this system addresses these challenges by providing consistent, detailed, and rapid evaluation.

Version 2.0 Updates:

- Partial answers contribute zero marks to final score
- Human-like, exam-specific feedback (not generic AI responses)
- Clear guidance for students to consult markers on partial answers
- Removal of vague, ChatGPT-style feedback
- Focus on specific missing concepts from marking scheme

1.2 Key Features

- **Multi-dimensional Analysis:** Evaluates answers across semantic similarity, keyword coverage, and concept understanding
- **Advanced NLP Models:** Utilizes Sentence-BERT, SpaCy, and TextBlob for comprehensive language analysis
- **Human-like Feedback:** Provides specific, exam-focused feedback without generic AI phrases
- **Partial Credit Policy:** Partial answers earn no marks; students directed to consult markers
- **Quality Metrics:** Analyzes writing quality including word count, lexical diversity, and sentence structure
- **PDF Report Generation:** Produces professional, comprehensive grading reports
- **Interactive UI:** User-friendly interface for easy interaction with the system

1.3 System Architecture

The system comprises four main components:

1. **Advanced Grading Engine:** Core evaluation logic using NLP models
2. **Feedback Generator:** Produces detailed, context-aware, human-like feedback
3. **PDF Report Generator:** Creates professional assessment reports
4. **Interactive Interface:** Web-based UI for user interaction

2 Technical Stack

2.1 Core Libraries and Dependencies

The system leverages several state-of-the-art NLP libraries:

Table 1: Core Libraries and Dependencies

Library	Version	Purpose
sentence-transformers	Latest	Semantic similarity analysis
transformers	Latest	BERT-based models
torch	Latest	Deep learning framework
spacy	3.x	Linguistic analysis
textblob	Latest	Sentiment and readability
scikit-learn	Latest	Cosine similarity calculations
nltk	Latest	Natural language toolkit
reportlab	Latest	PDF generation
ipywidgets	Latest	Interactive UI components

2.2 NLP Models

Sentence Transformer Model

- **Model:** all-MiniLM-L6-v2
- **Description:** A sentence embedding model that maps sentences to a 384-dimensional dense vector space. Optimized for semantic similarity tasks with excellent performance and reasonable computational requirements.
- **Use Case:** Calculating semantic similarity between student answers and model answers.

SpaCy Language Model

- **Model:** en_core_web_sm
- **Description:** English language model for SpaCy including vocabulary, syntax, and entities trained on web text.
- **Use Case:** Part-of-speech tagging, named entity recognition, dependency parsing, and lemmatization.

3 System Components

3.1 Advanced Grading Engine

The `AdvancedGradingEngine` class is the core component responsible for evaluating student answers using multiple NLP techniques.

3.1.1 Initialization

```
1 class AdvancedGradingEngine:
2     def __init__(self):
3         print("Loading AI models...")
4
5         # Load Sentence Transformer
6         self.sentence_model = SentenceTransformer('all-MiniLM-L6-v2')
7
8         # Load SpaCy for linguistic analysis
9         self.nlp = spacy.load('en_core_web_sm')
10
11        # Thresholds
12        self.semantic_threshold = 0.65
13        self.keyword_threshold = 0.60
14        self.partial_threshold = 0.45
15
16        print("AI models loaded successfully!")
```

Listing 1: Grading Engine Initialization

3.1.2 Key Concept Extraction

The system extracts key concepts from text using multiple techniques:

- **Named Entity Recognition (NER):** Identifies entities like persons, organizations, locations
- **Noun Chunks:** Extracts noun phrases representing concepts
- **Keywords:** Identifies important nouns, proper nouns, and verbs
- **Key Phrases:** Extracts verb-noun pairs representing actions and objects

```
1 def extract_key_concepts(self, text):
2     """Extract key concepts using NER and noun chunks"""
3     doc = self.nlp(text)
4
5     concepts = {
6         'entities': [(ent.text, ent.label_) for ent in doc.ents],
7         'noun_chunks': [chunk.text for chunk in doc.noun_chunks],
8         'keywords': [token.text for token in doc
9                     if token.pos_ in ['NOUN', 'PROPN', 'VERB']
10                    and not token.is_stop],
11        'key_phrases': []
12    }
13
14    # Extract important verb-noun pairs
15    for token in doc:
16        if token.pos_ == 'VERB':
17            children = [child for child in token.children
18                      if child.pos_ in ['NOUN', 'PROPN']]
19            for child in children:
```

```

20         concepts['key_phrases'].append(
21             f"{token.text} {child.text}"
22         )
23
24     return concepts

```

Listing 2: Concept Extraction Method

3.1.3 Semantic Similarity Calculation

The system calculates semantic similarity using Sentence-BERT embeddings and cosine similarity:

$$\text{Similarity}(A, B) = \cos(\theta) = \frac{\mathbf{E}_A \cdot \mathbf{E}_B}{\|\mathbf{E}_A\| \times \|\mathbf{E}_B\|} \quad (1)$$

where $\mathbf{E}_A$ and $\mathbf{E}_B$ are the embeddings of texts A and B respectively.

```

1 def calculate_semantic_similarity(self, text1, text2):
2     """Calculate semantic similarity using Sentence-BERT"""
3     embeddings1 = self.sentence_model.encode(
4         text1, convert_to_tensor=True
5     )
6     embeddings2 = self.sentence_model.encode(
7         text2, convert_to_tensor=True
8     )
9
10    similarity = util.pytorch_cos_sim(embeddings1, embeddings2)
11    return similarity.item()

```

Listing 3: Semantic Similarity Calculation

3.1.4 Answer Quality Analysis

The system analyzes multiple quality metrics:

Table 2: Answer Quality Metrics

Metric	Description
Word Count	Total non-punctuation words
Sentence Count	Number of sentences
Avg Sentence Length	Words per sentence
Unique Words	Count of distinct words
Lexical Diversity	Ratio of unique to total words
Contains Examples	Boolean flag for example usage

3.1.5 Keyword Coverage Analysis

The system checks keyword coverage using lemmatization to handle word variations:

```

1 def check_keyword_coverage(self, student_text, keywords):
2     """Check how many keywords are covered"""
3     if not keywords:
4         return 1.0, []
5
6     student_lower = student_text.lower()
7     student_doc = self.nlp(student_lower)
8     student_lemmas = set([token.lemma_ for token in student_doc])
9

```



```

10 matched = []
11 for keyword in keywords:
12     keyword_doc = self.nlp(keyword.lower())
13     keyword_lemmas = set([token.lemma_ for token in keyword_doc])
14
15     # Check for direct match or lemma match
16     if (keyword.lower() in student_lower or
17         keyword_lemmas.intersection(student_lemmas)):
18         matched.append(keyword)
19
20 coverage = len(matched) / len(keywords)
21 return coverage, matched

```

Listing 4: Keyword Coverage Check

3.1.6 Point Grading Algorithm

The grading algorithm combines multiple scores with weighted contributions:

$$\text{Combined Score} = 0.5 \times S_{\text{semantic}} + 0.3 \times S_{\text{keyword}} + 0.2 \times S_{\text{concept}} \quad (2)$$

where:

- S_{semantic} = Semantic similarity score (50% weight)
- S_{keyword} = Keyword coverage score (30% weight)
- S_{concept} = Concept overlap score (20% weight)

Classification Thresholds:

$$\text{Correct: } \text{Combined} \geq 0.65 \text{ AND Keyword} \geq 0.60 \quad (3)$$

$$\text{Partial: } 0.45 \leq \text{Combined} < 0.65 \quad (4)$$

$$\text{Incorrect: } \text{Combined} < 0.45 \quad (5)$$

3.2 Enhanced Feedback Generator (Human-like)

The `EnhancedFeedbackGenerator` class creates specific, exam-focused feedback without generic AI phrases.

3.2.1 Point-Specific Feedback Generation

The system generates three types of feedback based on performance:

Correct Answers Provides specific positive feedback:

- "Point X: Full marks awarded. Your answer correctly identifies and explains the required concept with appropriate terminology."
- Acknowledges specific matched keywords
- No generic phrases like "Excellent work!" or "Keep it up!"

```

1 if status == 'correct':
2     if semantic_score > 0.85 and keyword_coverage > 0.85:
3         feedback['explanation'] = (
4             f"Point {point_number}: Full marks awarded. "
5             f"Your answer correctly identifies and explains "
6             f"the required concept with appropriate terminology."
7         )
8     elif len(matched_keywords) >= 3:
9         feedback['explanation'] = (
10            f"Point {point_number}: Correct. You've covered "
11            f"the key elements, including "
12            f"{', '.join(matched_keywords[:3])}."
13        )

```

Listing 5: Correct Answer Feedback

Incorrect Answers Provides specific feedback on missing concepts:

- Clear statement: "Point X: No marks awarded"
- Lists specific missing concepts from marking scheme
- "Your answer went in a different direction than what the question was asking for"
- No vague suggestions like "try to understand" or "add examples"

```

1 elif status == 'incorrect':
2     feedback['explanation'] = (
3         f"Point {point_number}: No marks awarded. "
4         f"Your response does not address the specific "
5         f"requirement for this marking point."
6     )
7
8     if keyword_coverage < 0.3 and missing_keywords:
9         feedback['suggestion'] = (
10            f"You need to discuss: "
11            f"{', '.join(missing_keywords[:4])}. "
12            f"Your answer went in a different direction "
13            f"than what the question was asking for."
14        )

```

Listing 6: Incorrect Answer Feedback

Partial Answers - CRITICAL CHANGE Key Update: Partial answers contribute ZERO marks to the final score.

- Clear statement: "Point X: Partial credit only - NO MARKS awarded"
- Explicit note: "(partial answers don't count toward your total)"
- Direct students to consult marker: "Please see me to understand exactly what's missing for full marks"
- Specific guidance: "See me during consultation hours to review exactly what the marking scheme expects here"

```

1 else: # partial
2     feedback['explanation'] = (
3         f"Point {point_number}: Partial credit only - "
4         f"NO MARKS awarded (partial answers don't count "
5         f"toward your total)."
6     )
7
8     if keyword_coverage >= 0.5:
9         feedback['suggestion'] = (
10            f"You mentioned some relevant terms "
11            f"({'', '.join(matched_keywords[:3])}) but your "
12            f"explanation is incomplete or lacks precision. "
13            f"Please see me to understand exactly what's "
14            f"missing for full marks."
15        )
16    else:
17        feedback['suggestion'] = (
18            f"Your answer is too vague or doesn't directly "
19            f"address what this point requires. See me during "
20            f"consultation hours to review exactly what the "
21            f"marking scheme expects here."
22        )

```

Listing 7: Partial Answer Feedback

3.2.2 Overall Feedback Generation

The system generates exam-specific overall feedback:

```

1 def generate_overall_feedback(self, score, max_score,
2                               partial_points, points_analysis):
3     """Generate comprehensive overall feedback"""
4     percentage = (score / max_score) * 100
5     correct_count = sum(1 for p in points_analysis
6                         if p['status'] == 'correct')
7     partial_count = sum(1 for p in points_analysis
8                        if p['status'] == 'partial')
9     incorrect_count = sum(1 for p in points_analysis
10                        if p['status'] == 'incorrect')
11
12     # Opening based on performance
13     if percentage >= 85:
14         opening = (
15             f"You scored {score}/{max_score}. This is a strong "
16             f"performance showing good understanding of the material."
17         )
18     elif percentage >= 70:
19         opening = (

```

```
20         f"You scored {score}/{max_score}. You've grasped most "
21         f"of the key concepts but there's room to improve."
22     )
23
24     # Breakdown
25     breakdown = f"Breakdown: {correct_count} point(s) fully correct"
26     if partial_count > 0:
27         breakdown += (
28             f", {partial_count} point(s) partially correct "
29             f"(no marks given)"
30         )
31
32     # Specific advice for partial answers
33     advice = ""
34     if partial_count > 0:
35         advice += (
36             f" For the {partial_count} partial point(s), you were "
37             f"close but didn't earn the marks. These are "
38             f"opportunities where a bit more precision or "
39             f"completeness would have earned you credit. Please "
40             f"review these with me to understand the exact "
41             f"requirements."
42         )
43
44     return f"{opening} {breakdown}{advice}"
```

Listing 8: Overall Feedback Generation

3.3 Scoring System - KEY CHANGE

Critical Update: Only fully correct answers contribute to the final score.

```

1 def grade_answer(self, student_answer, model_answer, max_marks):
2     """Grade student answer - PARTIAL ANSWERS DON'T ADD TO SCORE"""
3     model_points = self.parse_model_answer(model_answer)
4
5     points_analysis = []
6     correct_count = 0
7     partial_count = 0
8
9     for point in model_points:
10        grading_result = self.engine.grade_point(
11            student_answer,
12            point['text'],
13            point['keywords']
14        )
15
16        # ONLY CORRECT ANSWERS GET MARKS
17        if grading_result['status'] == 'correct':
18            correct_count += 1
19        elif grading_result['status'] == 'partial':
20            partial_count += 1 # Count but don't add to score
21
22        # Calculate final score - only correct answers count
23        final_score = int((correct_count / len(model_points)) * max_marks)
24
25        return {
26            'score': final_score,
27            'max_score': max_marks,
28            'correct_points': correct_count,
29            'partial_points': partial_count, # For display only
30            # ... other fields
31        }

```

Listing 9: Score Calculation - Partial Gets Zero Marks

3.4 PDF Report Generator

The PDFReportGenerator class creates professional assessment reports using ReportLab.

3.4.1 Report Structure

The PDF report includes the following sections:

1. **Title and Header:** System name and assessment information
2. **Student Information:** Name, email, date, time, evaluator
3. **Score Summary:** Visual score display with note on partial marks
4. **Student Answer:** Complete student response
5. **Point-by-Point Assessment:** Detailed feedback for each marking point
6. **Overall Assessment:** Comprehensive feedback
7. **Strengths and Areas to Address:** Categorized feedback
8. **Footer:** Generation timestamp

4 Interactive User Interface

4.1 UI Components

The interactive interface includes:

1. **Header Section:** System branding and description
2. **Step 1 - Student Information:** Name and email input
3. **Step 2 - Question Setup:** Question text, max marks, marking scheme
4. **Step 3 - Student Answer:** Text area for student response
5. **Action Buttons:** Grade Answer, Download PDF, Reset
6. **Results Display:** Comprehensive grading results

4.2 Grading Process Flow

1. Student Information Input
2. Question & Marking Scheme Setup
3. Student Answer Input
4. AI Grading Process
5. Results Display (with partial marks note)
6. PDF Report Generation

Figure 1: System Workflow

4.3 Model Answer Format

The system expects model answers in a specific format:

- **Endpoint Markers:** Use [ENDPOINT] to separate marking points
- **Keyword Markers:** Use triple parentheses ((keyword)) to mark important terms

Example:

1. The price mechanism ((allocates)) resources through ((demand)) and ((supply)) interaction. [ENDPOINT]
2. Prices serve as ((signals)) to producers about consumer preferences. [ENDPOINT]

5 Technical Specifications

5.1 Model Parameters

Table 3: Grading Parameters and Weights

Parameter	Value	Purpose
Semantic Threshold	0.65	Minimum similarity for correct classification
Keyword Threshold	0.60	Minimum keyword coverage required
Partial Threshold	0.45	Minimum score for partial credit (no marks)
Semantic Weight	0.50	Contribution to combined score
Keyword Weight	0.30	Contribution to combined score
Concept Weight	0.20	Contribution to combined score
Embedding Dimension	384	Sentence-BERT vector size

5.2 Performance Metrics

The system evaluates performance across multiple dimensions:

- **Semantic Similarity:** Cosine similarity of sentence embeddings (0-1)
- **Keyword Coverage:** Ratio of matched keywords to total keywords (0-1)
- **Concept Overlap:** Intersection of key concepts divided by model concepts (0-1)
- **Combined Score:** Weighted average of all three metrics (0-1)

6 System Advantages

6.1 Benefits Over Traditional Grading

1. **Consistency:** Applies same criteria uniformly across all submissions
2. **Speed:** Grades answers in seconds vs. minutes for manual grading
3. **Objectivity:** Eliminates human bias and fatigue effects
4. **Human-like Feedback:** Provides specific, exam-focused feedback
5. **Scalability:** Can handle hundreds of submissions simultaneously
6. **Learning Support:** Identifies specific improvement areas
7. **Documentation:** Generates professional PDF reports automatically
8. **Clear Expectations:** Makes marking criteria transparent to students

6.2 Educational Value

- **Immediate Feedback:** Students receive instant assessment
- **Formative Assessment:** Supports learning through detailed feedback
- **Concept Identification:** Highlights specific knowledge gaps
- **Transparency:** Students understand why partial answers didn't earn marks
- **Self-Paced Learning:** Students can practice independently
- **Marker Consultation:** Directs students to seek clarification appropriately

7 Key Improvements in Version 2.0

7.1 Partial Marking Policy

Previous System:

- Partial answers earned 0.5 marks per point
- Generic feedback like "You're on the right track"
- Vague suggestions to "add more detail"

New System:

- Partial answers earn ZERO marks
- Clear statement: "NO MARKS awarded (partial answers don't count)"
- Direct students to consult marker for specific requirements
- Explains exactly what was missing without generic advice

7.2 Feedback Quality

Removed Generic Phrases:

- "Excellent work! You've demonstrated..."
- "Try to understand not just WHAT..."
- "Consider adding concrete examples..."
- "With focused study, you can improve..."

Added Specific Feedback:

- "Point 3: No marks awarded. Your response does not address..."
- "You need to discuss: demand, supply, shortage..."
- "Your answer went in a different direction than required"
- "See me to understand exactly what's missing for full marks"

7.3 Exam-Focused Language

The system now uses terminology familiar in educational settings:

- "Review the marking scheme"
- "This marking point requires..."
- "See what concept you left out"
- "Consult with me during office hours"
- "Check the question carefully"

8 Limitations and Future Work

8.1 Current Limitations

- **Diagram Recognition:** Cannot evaluate hand-drawn diagrams or figures
- **Mathematical Notation:** Limited support for complex equations
- **Context Sensitivity:** May miss nuanced arguments in complex topics
- **Language Support:** Currently English-only
- **Creative Responses:** Better suited for factual than creative answers

8.2 Proposed Enhancements

1. **Multi-Modal Analysis:** Integrate OCR for diagram evaluation
2. **Mathematical Processing:** Add LaTeX and equation recognition
3. **Contextual Understanding:** Implement advanced reasoning models
4. **Multilingual Support:** Extend to multiple languages
5. **Adaptive Learning:** Personalize thresholds based on student level
6. **Historical Tracking:** Monitor student progress over time
7. **Subject-Specific Models:** Fine-tune for different academic disciplines

9 Conclusion

The Advanced AI Answer Grading System Version 2.0 represents a significant advancement in automated educational assessment with human-like feedback capabilities. By combining state-of-the-art NLP models (Sentence-BERT, SpaCy) with specific, exam-focused feedback generation mechanisms, the system provides accurate, consistent, and educationally valuable evaluations.

The system's multi-dimensional analysis approach—incorporating semantic similarity, keyword coverage, and concept overlap—ensures comprehensive assessment that mimics human expert evaluation. The human-like feedback guides students toward improvement while maintaining efficiency benefits of automation.

9.1 Key Achievements

- **Accurate semantic understanding** through transformer-based models
- **Clear marking policy:** Partial answers = zero marks, with transparent reasoning
- **Human-like feedback generation** with exam-specific recommendations
- **Professional documentation** through automated PDF reports
- **User-friendly interface** for easy adoption in educational settings
- **Transparent assessment** that helps students understand marking criteria

9.2 Impact on Education

This system demonstrates the potential of AI to augment educational assessment while maintaining the human touch essential for effective teaching. By providing:

1. Instant, detailed feedback that matches teacher communication style
2. Clear guidance on when to consult markers for clarification
3. Specific identification of missing concepts from marking schemes
4. Transparent scoring that helps students understand expectations

The system frees educators to focus on higher-level teaching activities while providing students with immediate, specific feedback that supports their learning journey.

9.3 Version 2.0 Summary

The key innovation in Version 2.0 is the shift from generic AI-generated feedback to specific, human-like assessment communication:

Table 4: Version Comparison

Previous Version	Version 2.0
Partial = 0.5 marks	Partial = 0 marks
"Excellent work!"	"Point X: Full marks awarded"
"Try to add examples"	"You need to discuss: [specific concepts]"
Generic improvement tips	"See me to review what's required"
AI-style feedback	Exam-focused, teacher-like feedback

Acknowledgments

This project was developed as part of research into AI-assisted educational technology with focus on human-like feedback generation. Special thanks to the open-source communities behind Sentence-Transformers, SpaCy, NLTK, and other libraries that made this system possible.

The Version 2.0 improvements were designed based on feedback from educators emphasizing the need for specific, exam-focused assessment language rather than generic AI-generated comments.

Appendix A: Sample Code Snippets

A.1 Complete Grading Function

```
1 def grade_answer(self, student_answer, model_answer, max_marks):
2     """Grade student answer - PARTIAL ANSWERS DON'T ADD TO SCORE"""
3     # Parse model answer into marking points
4     model_points = self.parse_model_answer(model_answer)
5
6     if not model_points:
7         return None
8
9     # Analyze answer quality
10    quality_metrics = self.engine.analyze_answer_quality(
11        student_answer
12    )
13
14    points_analysis = []
15    correct_count = 0
16    partial_count = 0
17
18    # Grade each point
19    for point in model_points:
20        grading_result = self.engine.grade_point(
21            student_answer,
22            point['text'],
23            point['keywords']
24        )
25
26        feedback = self.feedback_gen.generate_point_feedback(
27            grading_result,
28            point['text'],
29            student_answer,
30            point['number']
31        )
32
33        point_result = {
34            'id': point['id'],
35            'status': grading_result['status'],
36            'expected': point['text'],
37            'semantic_score': grading_result['semantic_score'],
38            'keyword_coverage': grading_result['keyword_coverage'],
39            'matched_keywords': grading_result['matched_keywords'],
40            'explanation': feedback['explanation'],
41            'suggestion': feedback['suggestion']
42        }
```

```
43     points_analysis.append(point_result)
44
45     # ONLY CORRECT ANSWERS GET MARKS
46     if grading_result['status'] == 'correct':
47         correct_count += 1
48     elif grading_result['status'] == 'partial':
49         partial_count += 1
50
51 # Calculate final score - only correct answers count
52 final_score = int(
53     (correct_count / len(model_points)) * max_marks
54 )
55
56 # Generate overall feedback
57 overall_feedback = self.feedback_gen.generate_overall_feedback(
58     final_score,
59     max_marks,
60     partial_count,
61     points_analysis
62 )
63
64 # Identify strengths and improvements
65 strengths, improvements = (
66     self.feedback_gen.generate_strengths_and_improvements(
67         points_analysis
68     )
69 )
70
71 return {
72     'score': final_score,
73     'max_score': max_marks,
74     'correct_points': correct_count,
75     'partial_points': partial_count,
76     'points_feedback': points_analysis,
77     'overall_feedback': overall_feedback,
78     'strengths': strengths,
79     'improvements': improvements,
80     'avg_confidence': np.mean(
81         [p['semantic_score'] for p in points_analysis]
82     ),
83     'quality_metrics': quality_metrics
84 }
85
```

Listing 10: Complete Grading Workflow

A.2 Human-like Feedback Generation

```

1 def generate_point_feedback(self, grading_result, model_point,
2                             student_answer, point_number):
3     """Generate specific feedback for a point"""
4     status = grading_result['status']
5     semantic_score = grading_result['semantic_score']
6     keyword_coverage = grading_result['keyword_coverage']
7     matched_keywords = grading_result['matched_keywords']
8     missing_keywords = [
9         kw for kw in grading_result['model_concepts']['keywords'][:8]
10        if kw.lower() not in student_answer.lower()
11    ]
12
13    feedback = {'explanation': '', 'suggestion': ''}
14
15    if status == 'correct':
16        # Specific positive feedback
17        if semantic_score > 0.85 and keyword_coverage > 0.85:
18            feedback['explanation'] = (
19                f"Point {point_number}: Full marks awarded. "
20                f"Your answer correctly identifies and explains "
21                f"the required concept with appropriate terminology."
22            )
23        elif len(matched_keywords) >= 3:
24            feedback['explanation'] = (
25                f"Point {point_number}: Correct. You've covered "
26                f"the key elements, including "
27                f"{', '.join(matched_keywords[:3])}."
28            )
29        else:
30            feedback['explanation'] = (
31                f"Point {point_number}: This is acceptable and "
32                f"captures the main idea required for this mark."
33            )
34
35    elif status == 'incorrect':
36        # Specific feedback on what's missing
37        feedback['explanation'] = (
38            f"Point {point_number}: No marks awarded. "
39            f"Your response does not address the specific "
40            f"requirement for this marking point."
41        )
42
43    if keyword_coverage < 0.3 and missing_keywords:
44        feedback['suggestion'] = (
45            f"You need to discuss: "
46            f"{', '.join(missing_keywords[:4])}. "
47            f"Your answer went in a different direction than "
48            f"what the question was asking for."
49        )
50    elif semantic_score < 0.35:
51        feedback['suggestion'] = (
52            "Your response doesn't match what this marking "
53            "point requires. Review the question carefully to "
54            "understand what's being asked."
55        )
56    else:
57        feedback['suggestion'] = (
58            "You touched on related ideas but missed the "
59            "specific point. Look at the marking scheme and "
60            "see what concept you left out."
61        )

```

```
62
63     else: # partial
64         # Clear feedback that it's partial and why
65         feedback['explanation'] = (
66             f"Point {point_number}: Partial credit only - "
67             f"NO MARKS awarded (partial answers don't count "
68             f"toward your total)."
69         )
70
71     if keyword_coverage >= 0.5:
72         feedback['suggestion'] = (
73             f"You mentioned some relevant terms "
74             f"({', '.join(matched_keywords[:3])}) but your "
75             f"explanation is incomplete or lacks precision. "
76             f"Please see me to understand exactly what's "
77             f"missing for full marks."
78         )
79     elif semantic_score >= 0.55:
80         feedback['suggestion'] = (
81             "You're on the right track but haven't fully "
82             "developed the answer. The key concepts need to be "
83             "stated more explicitly. Consult with me to see "
84             "what would make this a complete answer."
85         )
86     else:
87         feedback['suggestion'] = (
88             "Your answer is too vague or doesn't directly "
89             "address what this point requires. See me during "
90             "consultation hours to review exactly what the "
91             "marking scheme expects here."
92         )
93
94     return feedback
```

Listing 11: Feedback Generation for Different Statuses

Appendix B: Usage Instructions

B.1 Installation

Step 1: Install Required Packages

```
!pip install sentence-transformers transformers torch
!pip install scikit-learn nltk spacy textblob reportlab ipywidgets
!python -m spacy download en_core_web_sm
```

Step 2: Download NLTK Data

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
nltk.download('stopwords')
```

B.2 Running the System

Step 1: Load Part 1 (Grading Engine and Feedback Generator)

Step 2: Load Part 2 (PDF Generator and UI)

Step 3: Use the interactive interface:

1. Enter student information (name, email)
2. Load sample question or enter your own marking scheme
3. Enter student answer
4. Click "Grade Answer"
5. Review results and download PDF report

B.3 Marking Scheme Format

Format Requirements:

- Separate each marking point with [ENDPOINT]
- Mark important keywords with (((keyword)))
- Number points for easy reference

Example:

1. Resources are allocated through the ((price mechanism)) where ((demand)) and ((supply)) interact. [ENDPOINT]
2. Price increases serve as ((signals)) to producers indicating consumer demand. [ENDPOINT]
3. Higher prices provide ((incentives)) for firms to increase production and earn higher ((profits)). [ENDPOINT]

Appendix C: Example Assessment

C.1 Sample Question

(a) *Explain the role of prices in a free market economy. [10 marks]*

C.2 Sample Student Answer (Partial Quality)

"Prices help the economy work. When prices go up, people buy less things and companies make more. The supply and demand curves show this. Prices also help decide what to produce. If something is expensive, more will be made."

C.3 Expected Feedback

Score: 3/10

Point-by-Point:

- Point 1 (Price mechanism): **Partial - NO MARKS**
 - You mentioned demand and supply but didn't explain how the price mechanism allocates resources. See me to understand what's required.
- Point 2 (Signaling role): **Incorrect - 0 marks**
 - You didn't explain the signaling function of prices. You need to discuss: signals, producers, consumer preferences.
- Point 3 (Incentive role): **Correct - 1 mark**
 - Correct. You've covered the incentive aspect, including profits and production increases.

Overall Assessment:

You scored 3/10. Your response shows partial understanding but significant gaps remain. Breakdown: 3 points fully correct, 2 points partially correct (no marks given), 5 points incorrect. For the 2 partial points, you were close but didn't earn the marks. Please review these with me to understand the exact requirements.

This system provides transparent, consistent, and educationally valuable automated assessment with human-like feedback.

Version 2.0 - December 2024